



HACKTHEBOX



Lesson

26th September 2023 / Document No. D23.102.121

Prepared By: `w3th4nds`

Challenge Author(s): `w3th4nds`

Difficulty: **Easy**

Classification: Official

Synopsis

- Lesson is an easy difficulty challenge that features analyzing C code and answering some questions to get the flag.

Description

- It's time to learn some basic things about binaries and basic c. Answer some questions to get the flag.

Skills Required

- Basic C source code auditing

Skills Learned

- Familiarize with C source code analysis

Enumeration

We are given a `.zip` file that contains a `README.txt`, a `binary`, its `source code` and a `glibc` folder.

```
challenge git:main > l

glibc/  main*  main.c  README.txt
```

Reading `README.txt` tells us that we don't need to pwn the actual binary, just answer some questions based on it.

For this challenge, you do not have to pwn the given binary to get the flag.
Answer the questions based on the program and you will be rewarded with the flag.
Connect to the remote instance with: `$ nc` e.g. `nc localhost 1337`

The source code is this:

```
#include <stdio.h>

void under_construction(){
    printf("This is under development\n");
}

void print_msg(char *user){
    char formatter[0x20];
    strncpy(formatter, user, 5);
    for (size_t i = 0; i < 5; i++) formatter[i] = tolower(formatter[i]);
    printf(strncmp(formatter, "admin", 5) == 0 ? "\nWelcome admin!\n\n" : "\nWelcome
user!\n\n");
}

int main(int argc, char **argv){
    char name[0x20] = {0};
    unsigned long x, y;
    printf("Enter your name: ");
    scanf("%s", name);
    print_msg(name);
    return 0;
}
```

Now, connecting to the remote instance we see some **HINTS** and the corresponding question.

[illegible]

Is this a '32-bit' or '64-bit' ELF? (e.g. 1337-bit)

Is this a '32-bit' or '64-bit' ELF? (e.g. 1337-bit)

We open the debugger as the program prompts us and type `checksec`.

```
pwndbg> checksec
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
RUNPATH:   b'./glibc/'
```

From the 3 protections mentioned, only **NX** is enabled.

Question 3

HINT: Pay attention to the 'void print_msg(char *user)'
and the 'strcmp(arg1, arg2)'.

[*] Question number 0x3:

What do you need to enter so the message 'Welcome admin!' is printed?

```
[*] Question number 0x3:
```

What do you need to enter so the message 'Welcome admin!' is printed?

Taking a look at this function:

```
void print_msg(char *user){
    char formatter[0x20];
    strncpy(formatter, user, 5);
    for (size_t i = 0; i < 5; i++) formatter[i] = tolower(formatter[i]);
    printf(strncmp(formatter, "admin", 5) == 0 ? "\nWelcome admin!\n\n" : "\nWelcome
user!\n\n");
}
```

It takes our input and converts it to lowercase. Then, it compares the first 5 bytes of the input with "admin". So, either we give ADMIN, Admin, admin and so on, the program will print the message.

Question 4

[illegible]

```
[*] Question number 0x4:
```

What is the size of the 'name' buffer (in hex or decimal)?

As we can see from the image, the buffer is `0x20` or `32` bytes long.

Question 5

[illegible]

```
[*] Question number 0x5:
```

Which custom function is never called? (e.g. `vuln()`)

Taking a look at the `C` code again:

```
#include <stdio.h>

void under_construction(){
    printf("This is under development\n");
}

void print_msg(char *user){
    char formatter[0x20];
    strncpy(formatter, user, 5);
    for (size_t i = 0; i < 5; i++) formatter[i] = tolower(formatter[i]);
    printf(strncmp(formatter, "admin", 5) == 0 ? "\nWelcome admin!\n\n" : "\nWelcome
user!\n\n");
}

int main(int argc, char **argv){
    char name[0x20] = {0};
    unsigned long x, y;
    printf("Enter your name: ");
    scanf("%s", name);
    print_msg(name);
    return 0;
}
```

The only function never called is `under_construction()`.

```
void under_construction(){
    printf("This is under development\n");
}
```

Question 6

```
.....
●
● HINT: Which function reads the string from the stdin? ●
●
.....
```

[*] Question number 0x6:

What is the name of the standard function that could trigger a Buffer Overflow? (e.g. `fprintf()`)

The function that reads from the `stdin` in our program, is `scanf()`, thus it's the only one that could trigger a Buffer Overflow.

Question 7

```
.....
●
● HINT: A Segmentation Fault occurs when the return ●
● address is overwritten with an invalid address. ●
●
.....
```

[*] Question number 0x7:

Insert 30, then 39, then 40 'A's in the program and see the output.

After how many bytes a Segmentation Fault occurs (in hex or decimal)?

After entering 40 "A"s, we that the program stops with a `Segmentation Fault` error. If we enter 39 "A"s, it does not produce this error.

```
challenge git:main > ./main
```

```
Enter your name: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Welcome user!
```

```
[1] 56606 segmentation fault (core dumped) ./main
```

```
challenge git:main > ./main
```

```
Enter your name: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Welcome user!
```

```
challenge git:main >
```

Question 8

```

●
● HINT: Run 'gdb ./main' to open the binary in the debugger, then
● run 'p <function_name>' to see the address of a function.
●
● e.g. pwndbg> p main
● $2 = {<text variable, no debug info>} 0x401294 <main>
●

```

```
[*] Question number 0x8:
```

What is the address of 'under_construction()' in hex? (e.g. 0x401337)

We once again open the debugger and follow the instructions to get the address of `under_construction()`.

```
pwndbg> p under_construction
$1 = {<text variable, no debug info>} 0x4011d6 <under_construction>
```

We see that the address of `under_construction()` is `0x4011d6`. This was the last question and after that we get the flag.

```
[*] Question number 0x8:
```

What is the address of 'under_construction()' in hex? (e.g. 0x401337)

```
>> 0x4011d6
```

Correct

Great job! It's high time you solved your first challenge! Here is the flag!

HTB{REDACTED}